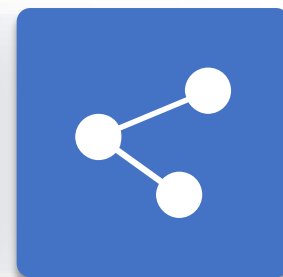




人工智能引论

第3章 状态空间与无信息搜索

授课教师：李静瑶





录

- 1 问题求解智能体
- 2 问题实例
- 3 搜索算法
- 4 无信息搜索策略



- 无信息搜索——盲目搜索

- 除了问题定义中提供的状态信息之外，没有任何附加信息
- 搜索算法只需生成后继、区分目标状态与非目标状态，按照节点扩展顺序分类

- 有信息搜索——启发式搜索

- 搜索算法知道一个状态是否比其它状态“更有希望”接近目标

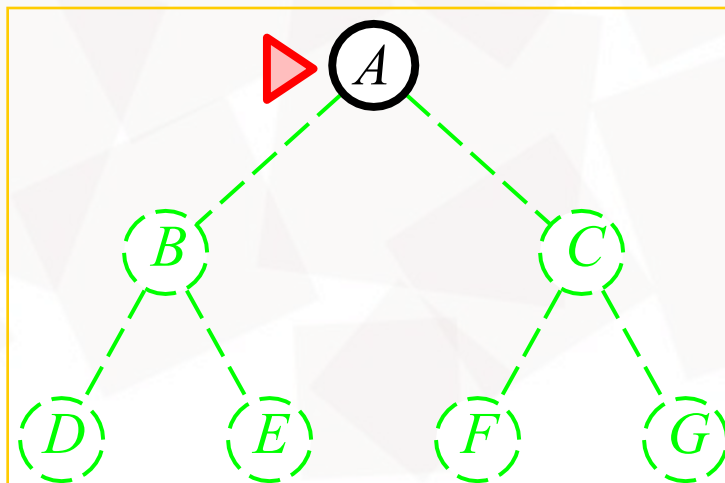


无信息搜索

- 广度优先搜索 (宽度优先搜索)
- 一致代价搜索
- 深度优先搜索
- 深度受限搜索
- 迭代加深搜索
- 双向搜索

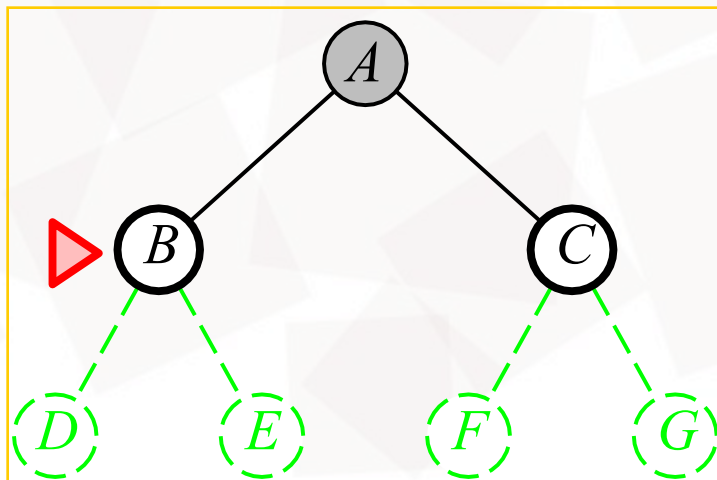
广度优先搜索

- 扩展最浅层的没有被扩展的节点



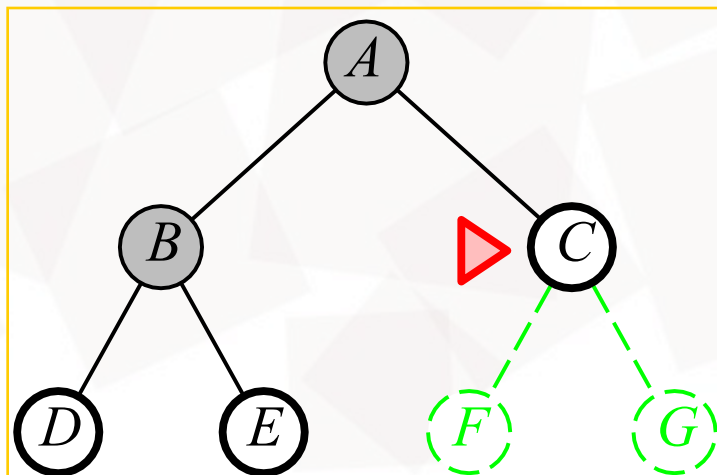
广度优先搜索

- 扩展最浅层的没有被扩展的节点



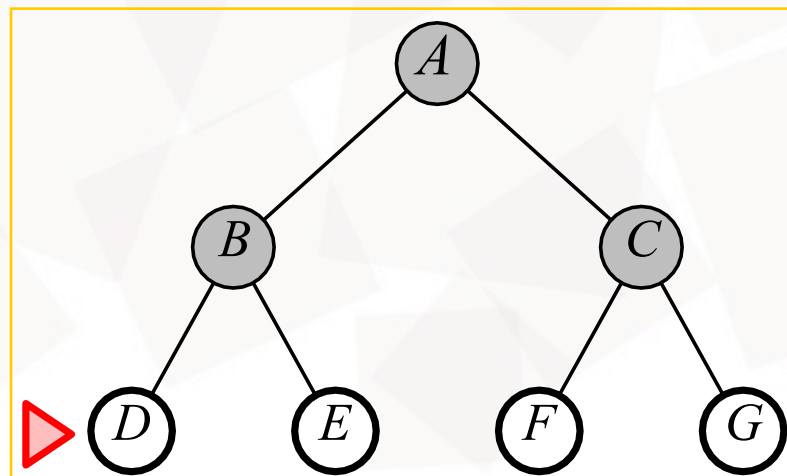
广度优先搜索

- 扩展最浅层的没有被扩展的节点



广度优先搜索

- 扩展最浅层的没有被扩展的节点



广度优先搜索

function BREADTH-FIRST-SEARCH(*problem*) **returns** 一个解节点或 *failure*

node $\leftarrow$ NODE(*problem*.INITIAL)

if *problem*.Is-GOAL(*node*.STATE) **then return** *node*

边界 *frontier* $\leftarrow$ 一个FIFO队列, 其中一个元素为 *node*

已达 *reached* $\leftarrow$ {*problem*.INITIAL}

while not Is-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *problem*.Is-GOAL(*s*) **then return** *child*

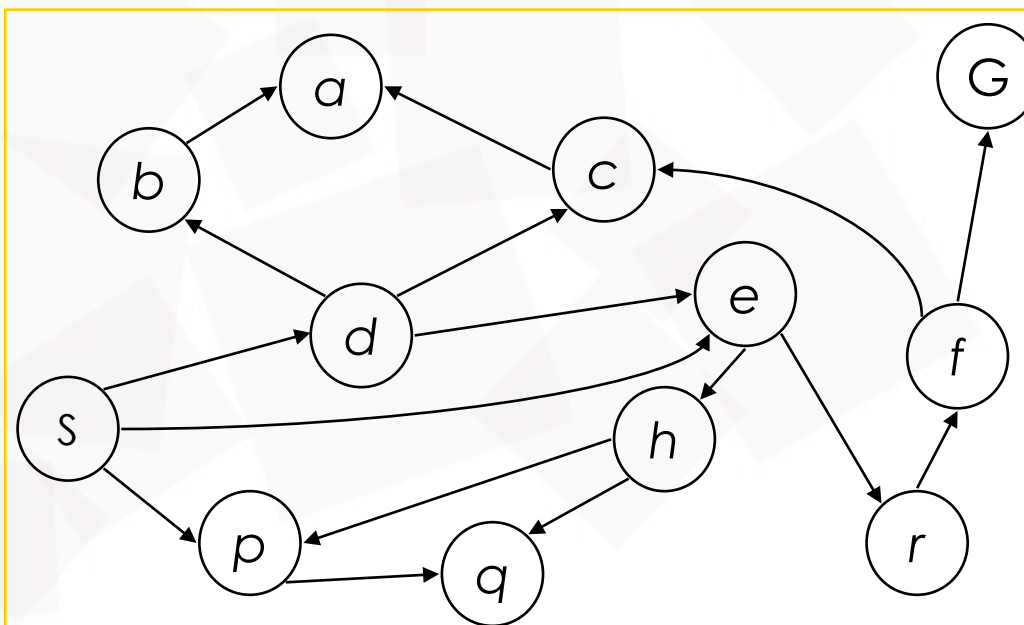
if *s* 不在 *reached* 中 **then**

将 *s* 添加到 *reached*

将 *child* 添加到 *frontier*

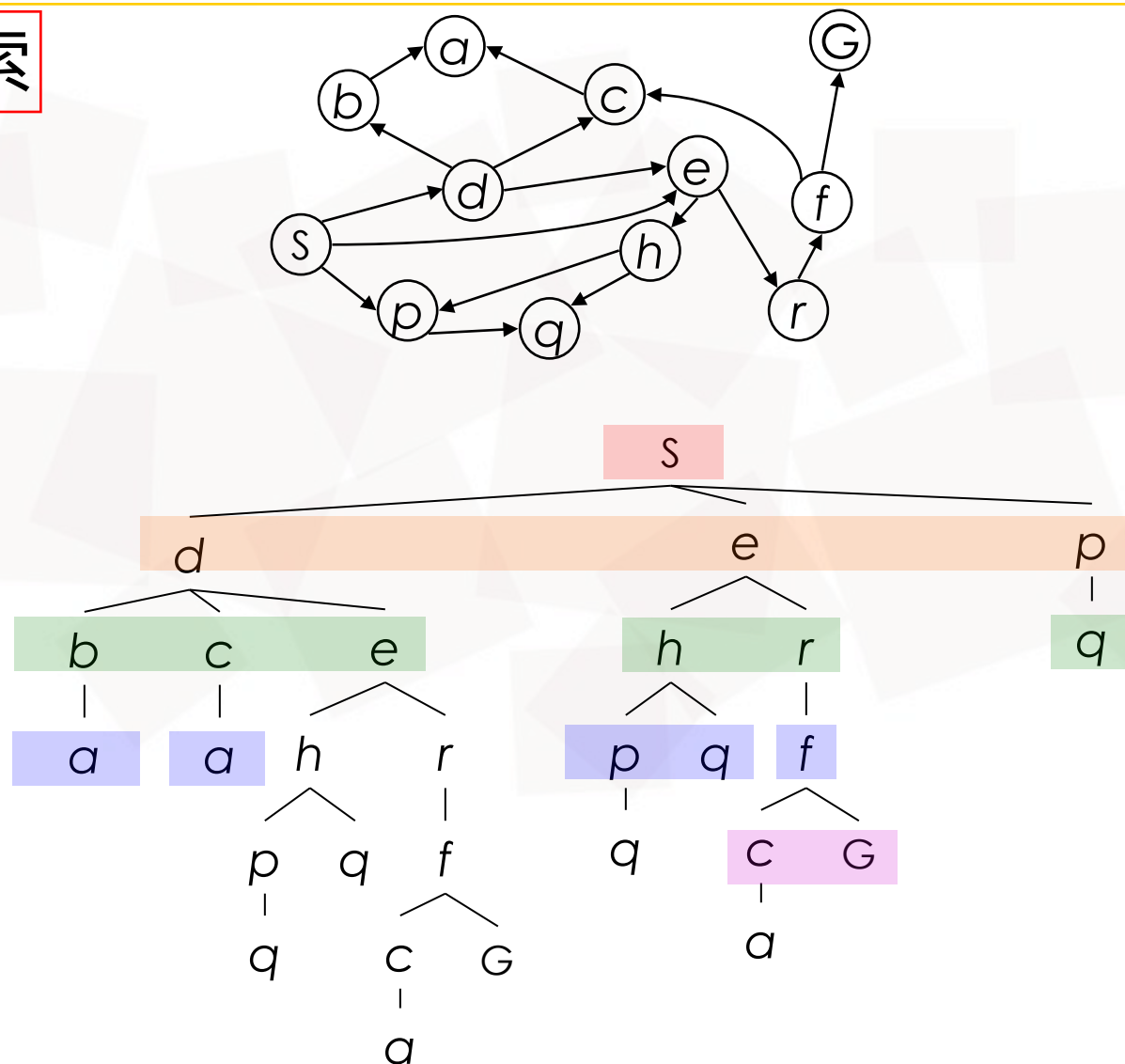
return *failure*

已达: 任何已经生成过节点的状态都称为已达 (reached) 状态 (无论该节点是否被扩展)



广度优先搜索

搜索层





广度优先搜索

- 完备性：算法完备
- 最优性：当动作代价相同时，算法最优

- 时间复杂度：

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

- 空间复杂度： $O(b^d)$ 个节点

(b -分支因子, d -搜索深度, 目标节点深度)



广度优先搜索的时空代价

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

图 3.13 宽度优先搜索的时间和内存代价。假设：分支因子为 $b = 10$ ；1 000 000 个结点/秒；1000 字节/结点

一般来说，指数级复杂性的搜索问题无法通过无信息搜索求解

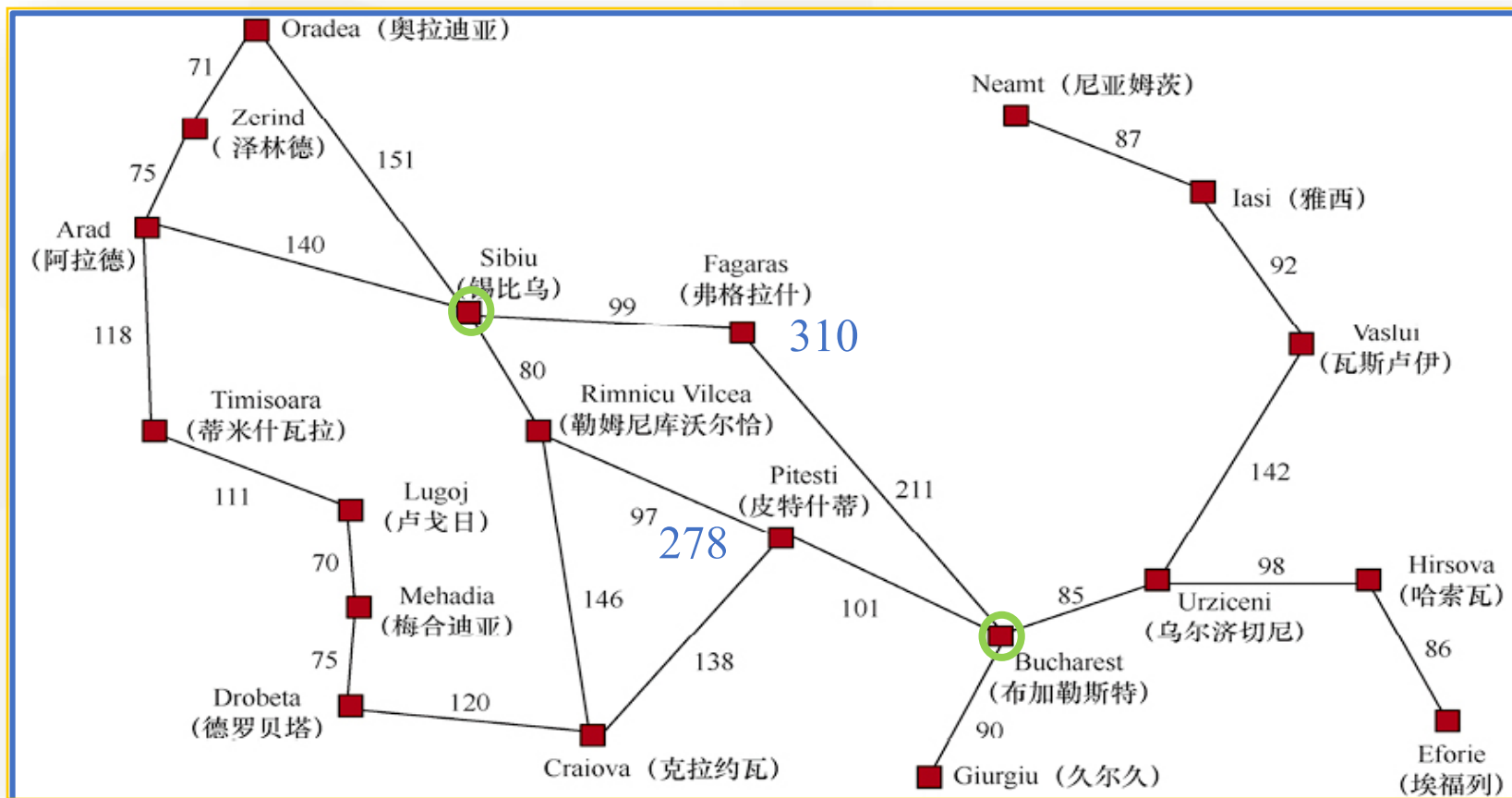


一致代价搜索

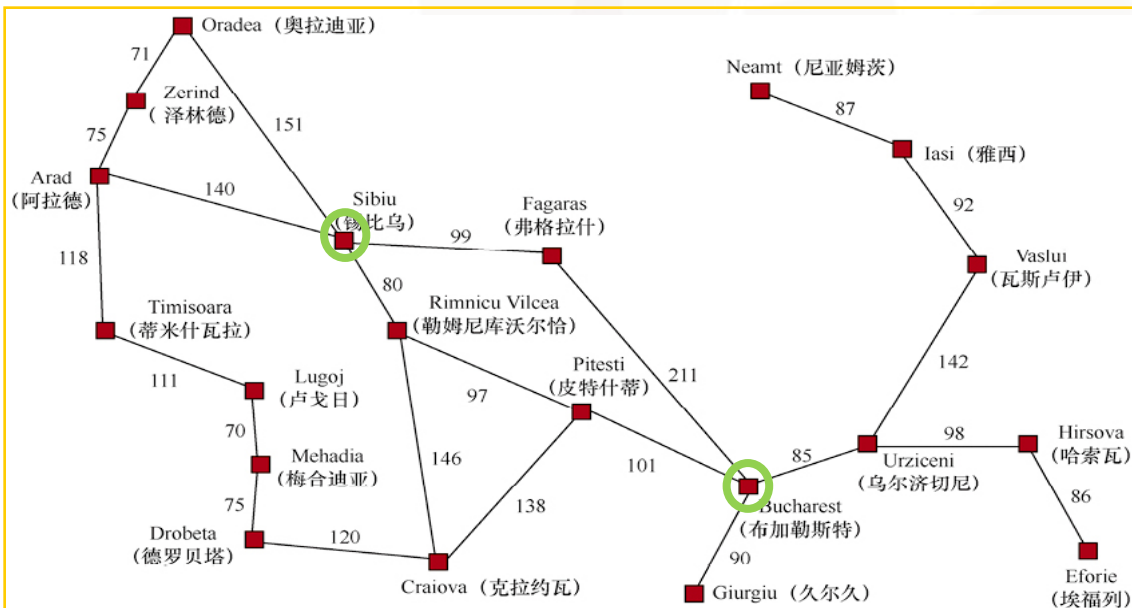
- 扩展路径代价最小的节点

```
function UNIFORM-COST-SEARCH(problem) returns 一个解节点或failure  
return BEST-FIRST-SEARCH(problem, PATH-COST)
```

一致代价搜索



一致代价搜索



function BEST-FIRST-SEARCH(*problem*, *f*) **returns** 一个解节点或 *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ 一个以 *f* 排序的优先队列，其中一个元素为 *node*

reached $\leftarrow$ 一个查找表，其中一个条目的键为 *problem*.INITIAL，值为 *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* in EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* 不在 *reached* 中 **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

将 *child* 添加到 *frontier* 中 (添加或替换)

return *failure*

function EXPAND(*problem*, *node*) **yields** 节点

s $\leftarrow$ *node*.STATE

for each *action* in *problem*.ACTIONS(*s*) **do**

s' $\leftarrow$ *problem*.RESULT(*s*, *action*)

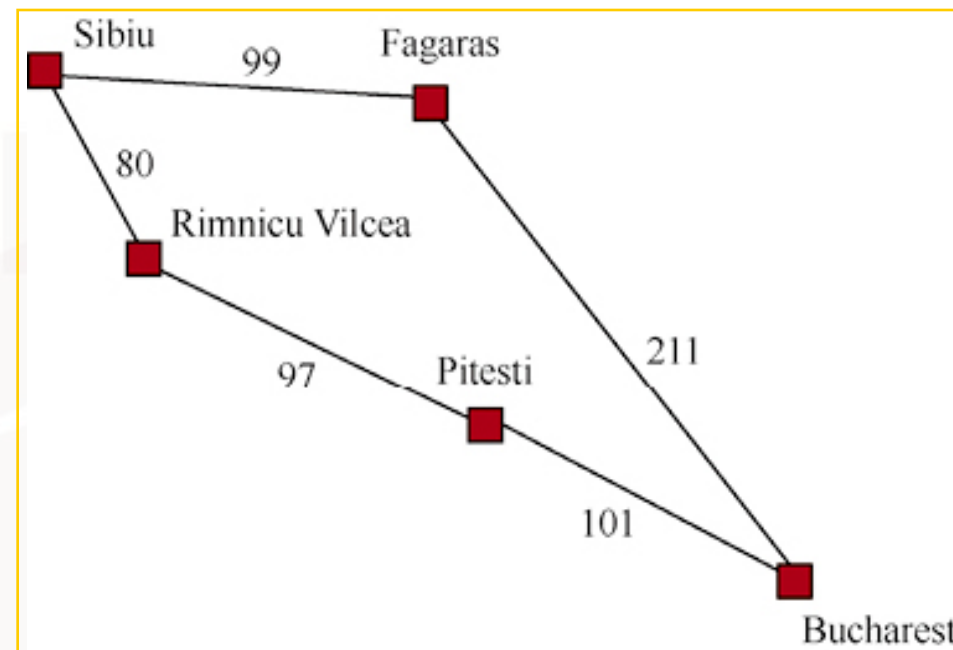
cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)



一致代价搜索

- 扩展S, 生成R[80]、F[99]
- 扩展R, 生成P[$80+97=177$]
- 扩展F, 生成B[$99+211=310$]
- 扩展P, 生成B[$177+101=278$]
- 更新路径, 保留B[278]
- 扩展B, 目标结点, 算法结束
- 结点S到B的路径代价值为278



一致代价搜索

```

function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node  $\leftarrow$  a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier  $\leftarrow$  a priority queue ordered by PATH-COST, with node as the only element
  explored  $\leftarrow$  an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node  $\leftarrow$  POP(frontier) /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child  $\leftarrow$  CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier  $\leftarrow$  INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
  
```

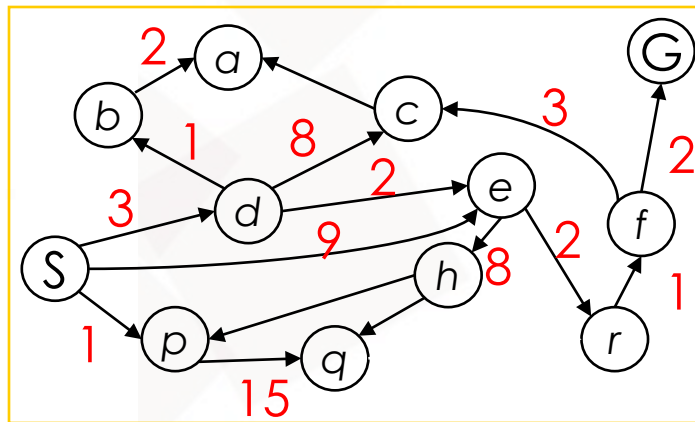
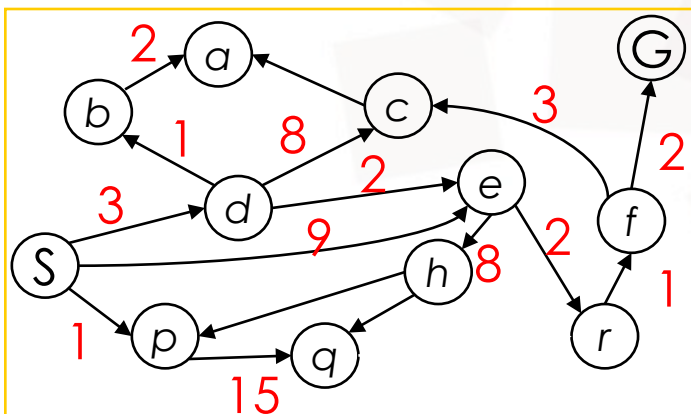
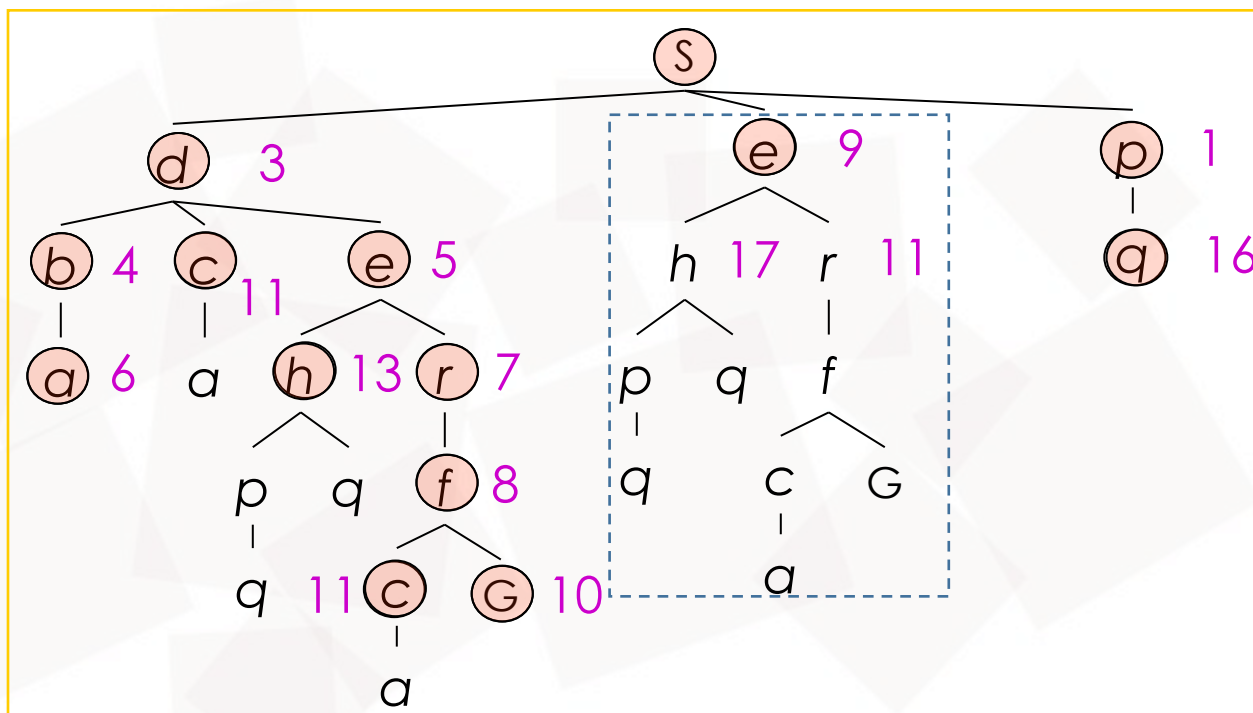
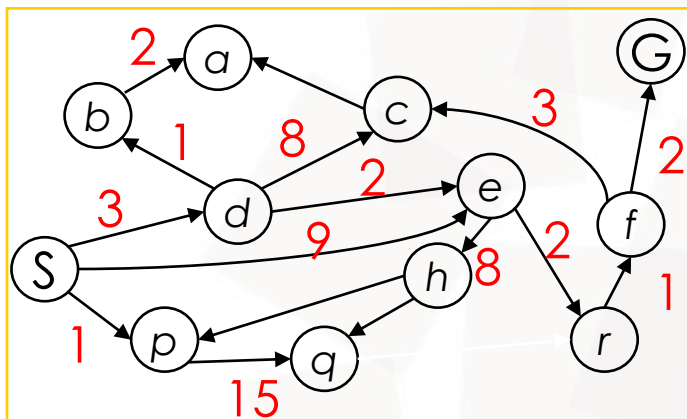


图 3.14 图的一致代价搜索。算法与图 3.7 给出的一般图搜索算法有不同，它使用了优先级队列并在边缘中的状态发现更小代价的路径时引入了额外的检查。边缘的数据结构需要支持有效的成员检测，这样它就结合了优先级队列和哈希表的能力



一致代价搜索



e的代价为9的节点会被更新成代价为5，
从而代价为9的e节点不会被扩展



一致代价搜索

- 完备性

- 若存在代价值为0的动作NOOP，则可能陷入死循环，算法不完备
- 若 b 有限且动作的代价值均不为0，则算法完备

- 最优性：最优， C^* 表示最优解路径的代价值

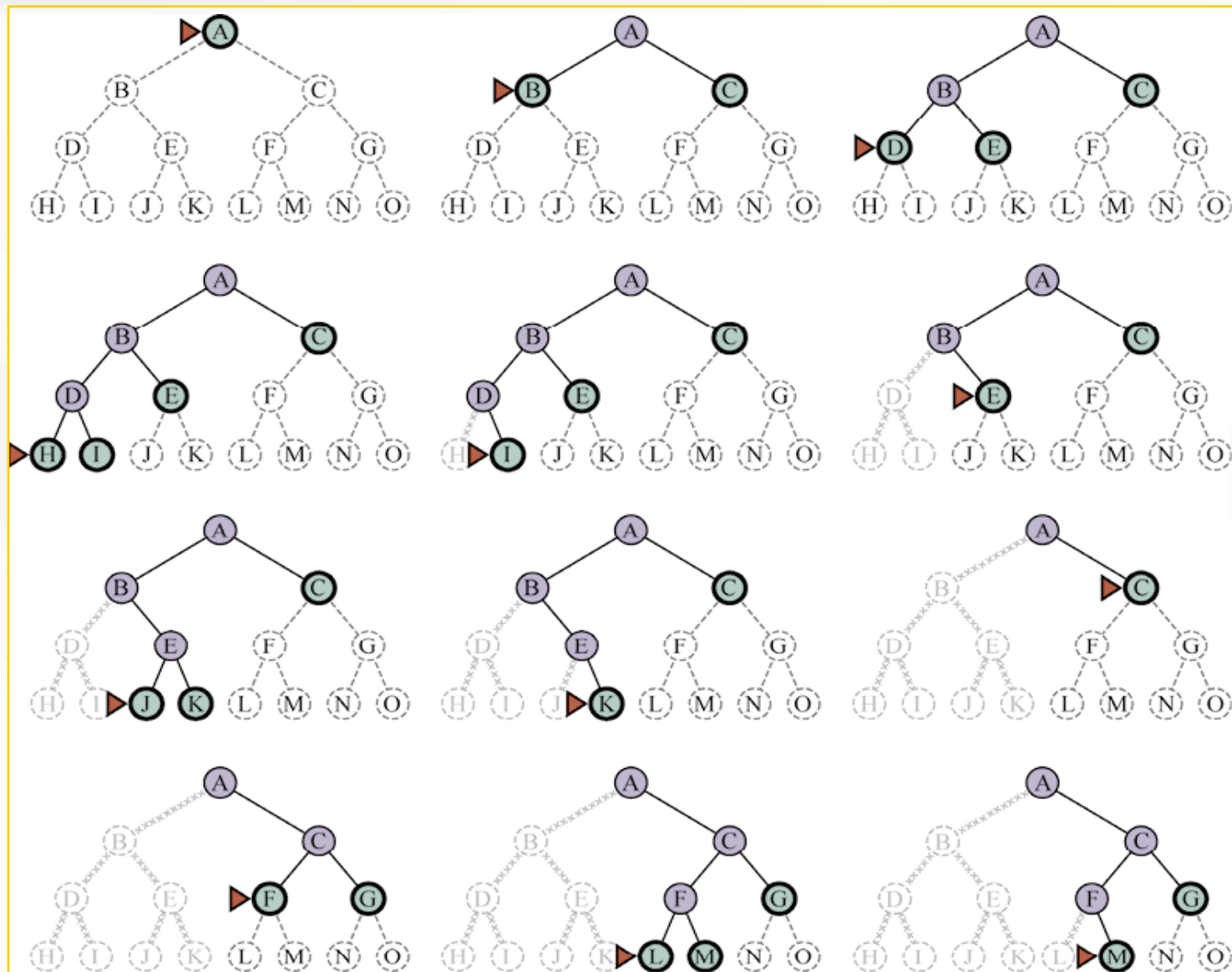
- 时间复杂度

- 一致代价搜索 > 宽度优先搜索

- 当动作代价值相同时，除了目标测试的时间点不同之外，一致代价搜索退化为宽度优先搜索

深度优先搜索

- 扩展最深层的
没有被扩展的
节点

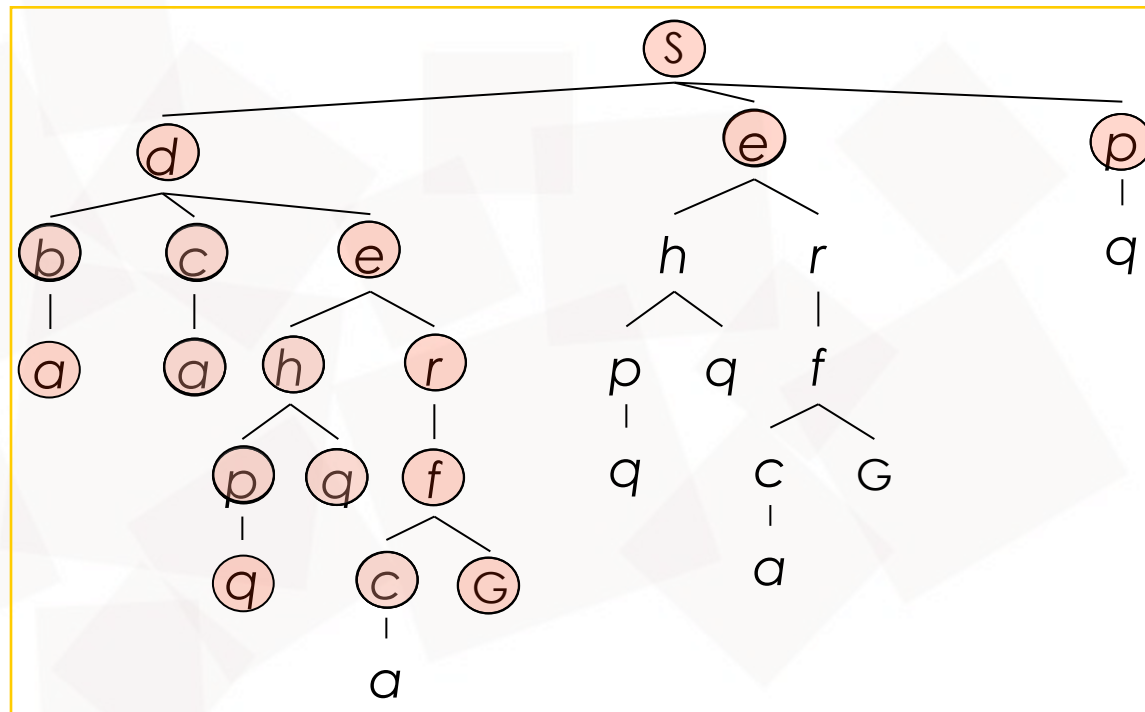
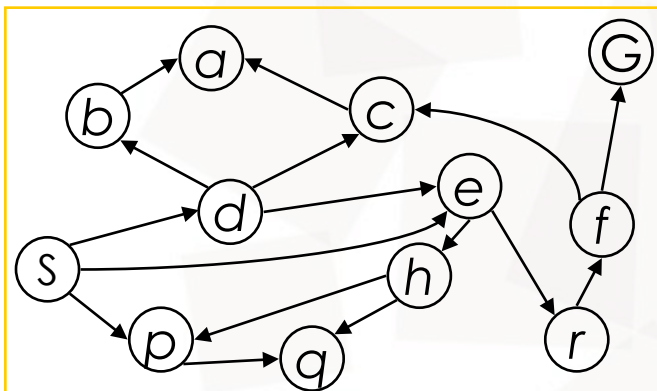


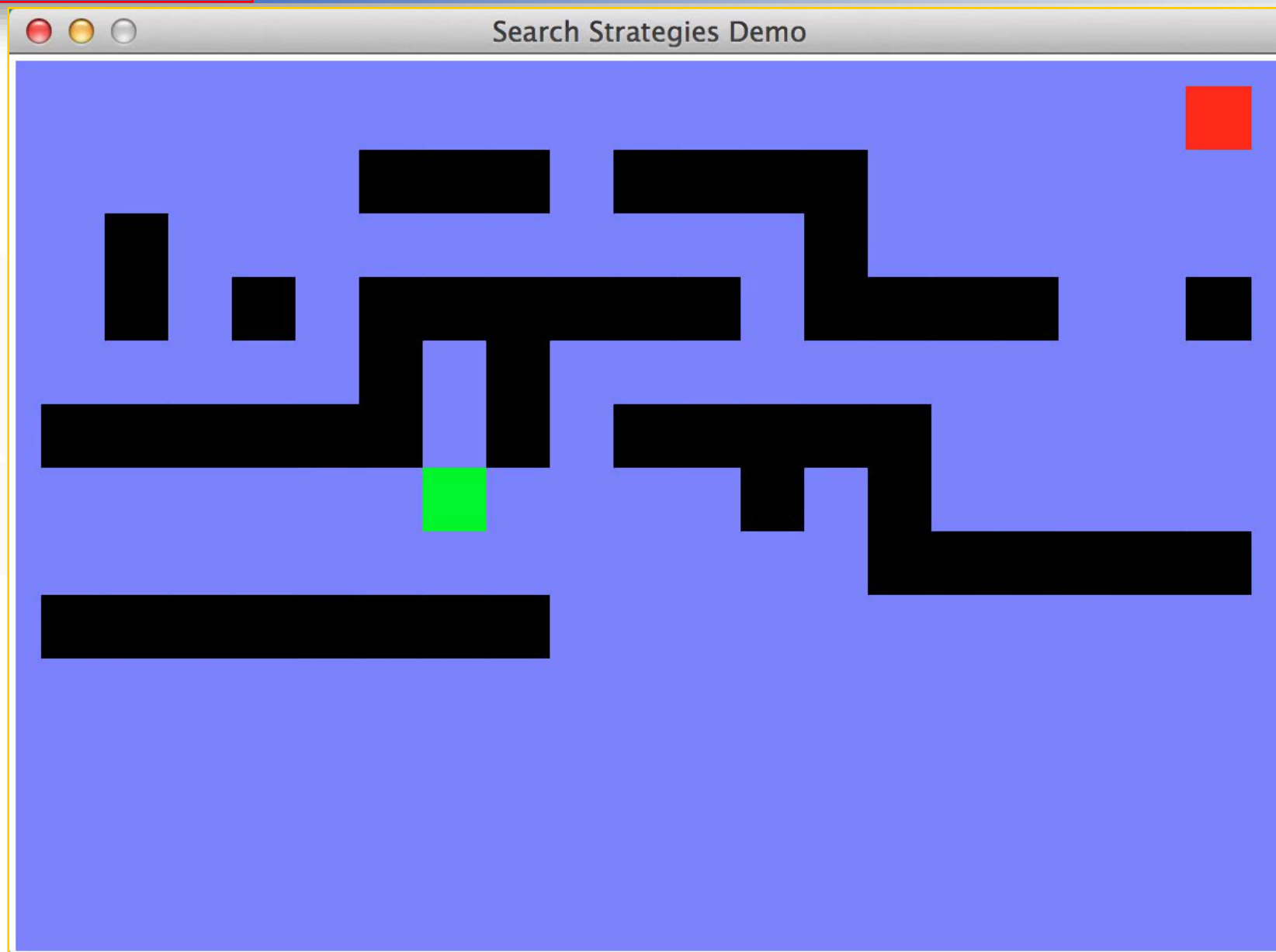
深度优先搜索 (图搜索)

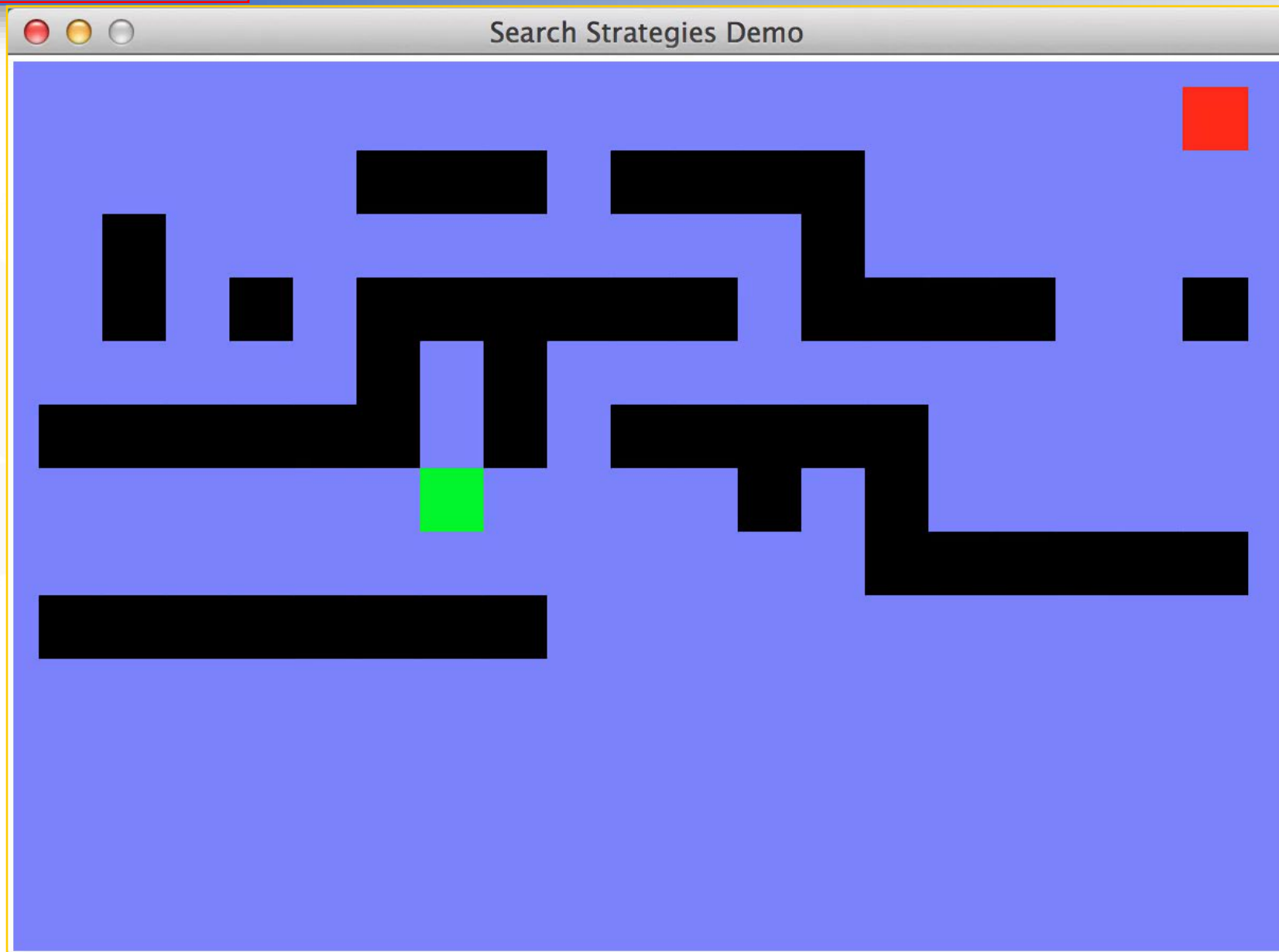
```
function DEPTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a LIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier) /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

图 3.11 图的深度优先搜索

深度优先搜索 (树搜索)









深度优先搜索

- 完备性
 - 有限状态空间上的图搜索，算法完备
 - 无限状态空间上的图搜索，算法不完备，可能存在无限且无法到达目标结点的路径（例：Donald Knuth猜想，一直执行阶乘动作）
 - 树搜索，算法不完备，可能陷入死循环
- 最优性：非最优



深度优先搜索

- 时间复杂度

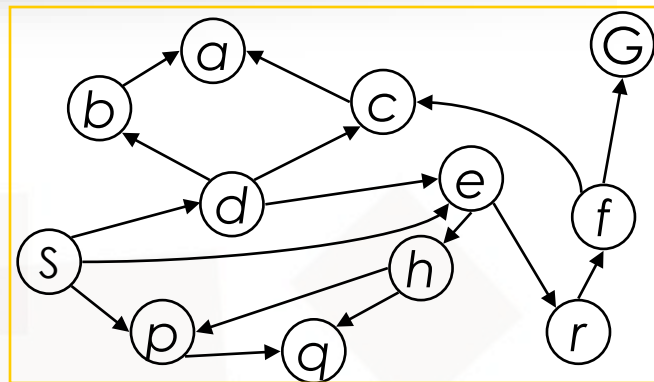
- 树搜索，最多生成 $O(b^m)$ 个结点，可能大于状态空间的规模， m 可能是无限的（ m -节点的最大深度）

- 空间复杂度（优势）

- 当一个结点的所有子结点均被探索，则该结点可以删除
- 仅需存储一条从根结点到叶结点的路径，以及该路径上每个结点的所有未被扩展的兄弟结点， $O(bm)$

回溯搜索

- 每次扩展只生成一个子节点，而不是所有子节点
- 空间复杂度
 - 每个被部分扩展的父节点要记住下一个要生成的子节点，保存 $O(m)$ 个节点
 - 通过直接修改父节点生成子节点，保存1个初始状态描述和 $O(m)$ 个动作
 - 为了回溯生成下一个子节点，必须能够撤销每次修改





深度受限搜索

- 添加了深度界限 ℓ : 深度为 ℓ 的节点当做叶节点对待
- 完备性: 当 $\ell < d$ 时, 算法不完备
- 时间复杂度: $O(b^\ell)$
- 空间复杂度: $O(b^\ell)$
- 当 $\ell = \infty$ 时, 深度受限搜索退化为深度优先搜索



迭代加深搜索

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** 一个解节点或*failure*

for *depth* = 0 **to** ∞ **do**

result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)

if *result* $\neq$ *cutoff* **then return** *result*

function DEPTH-LIMITED-SEARCH(*problem*, ℓ) **returns** 一个解节点或*failure*或*cutoff*

frontier $\leftarrow$ 一个LIFO队列（栈），其中一个元素为NODE(*problem*.INITIAL)

result $\leftarrow$ *failure*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

if DEPTH(*node*) $>$ ℓ **then**

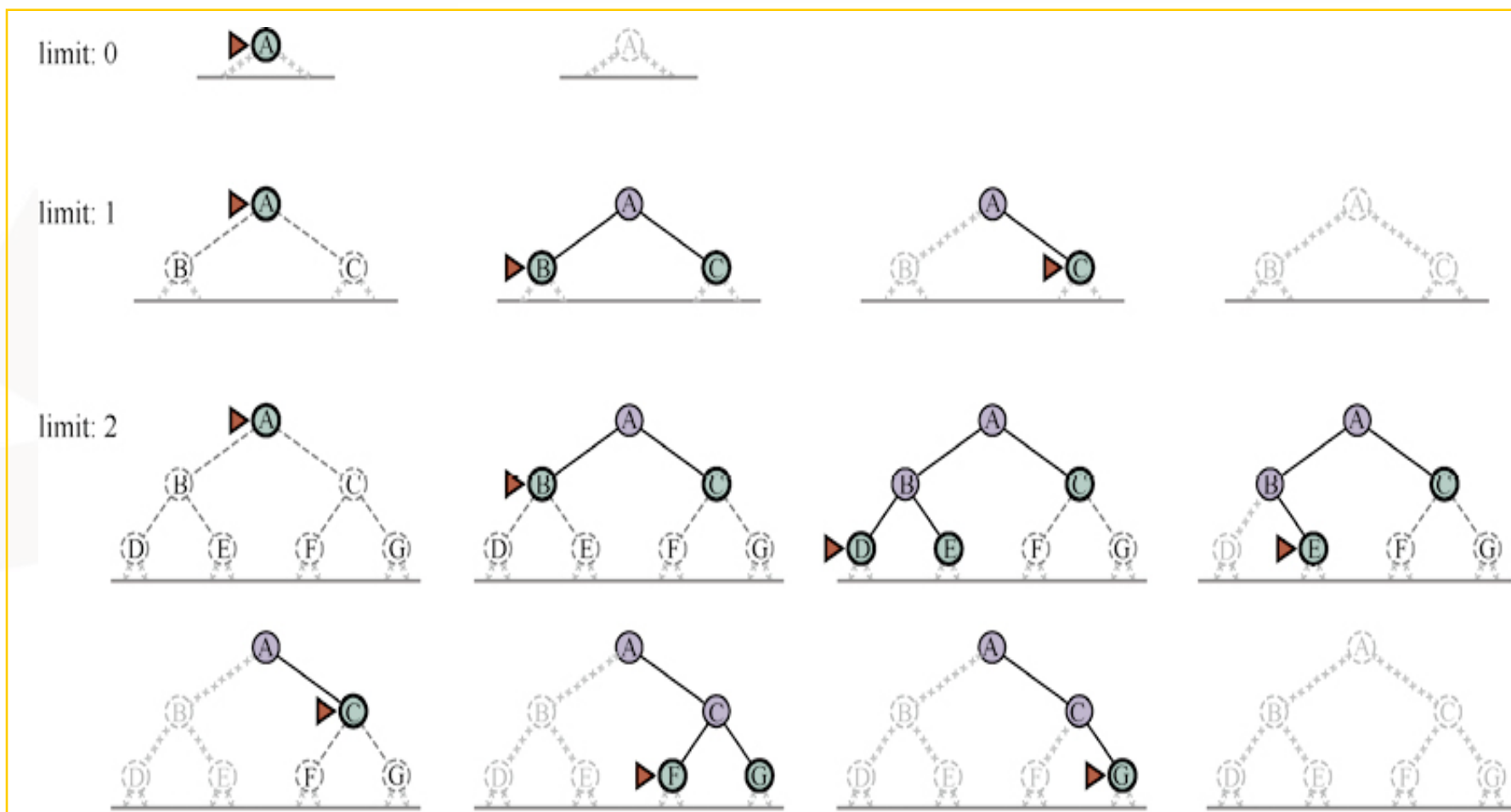
result $\leftarrow$ *cutoff*

else if not IS-CYCLE(*node*) **do**

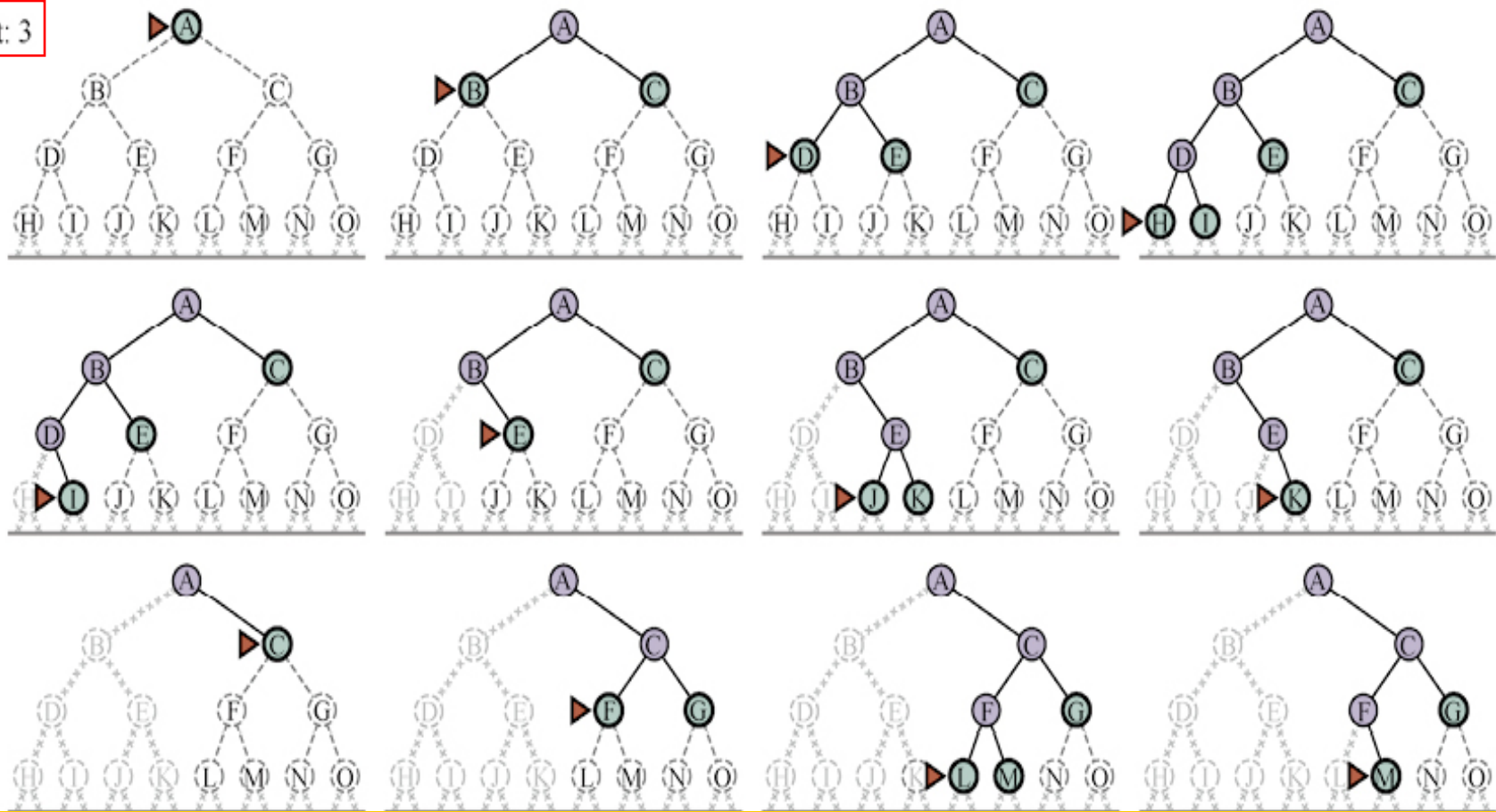
for each *child* **in** EXPAND(*problem*, *node*) **do**

将*child*添加到*frontier*

return *result*



limit: 3





迭代加深搜索

- 完备性：若 b 有限，则算法完备
- 最优性：
 - 若路径代价是节点深度的非递减函数，则算法最优
 - 当动作代价值相同时，算法最优
- 时间复杂度： $(d)b^1 + (d-1)b^2 + (d-2)b^3 + \dots + b^d = O(b^d)$
- 空间复杂度： $O(bd)$



双向搜索

- 两个搜索同时运行：一个从初始状态向前搜索，另一个从目标状态向后搜索，直到状态相遇
- 目标测试：检查两个搜索的边界集是否相交
- 优点：搜索区域比深度优先搜索小
- 限制：只能用在支持反向搜索的场合
- 最优性：非最优
- 双向宽度优先搜索的时间复杂度： $O(b^{d/2})$
- 双向宽度优先搜索的空间复杂度： $O(b^{d/2})$

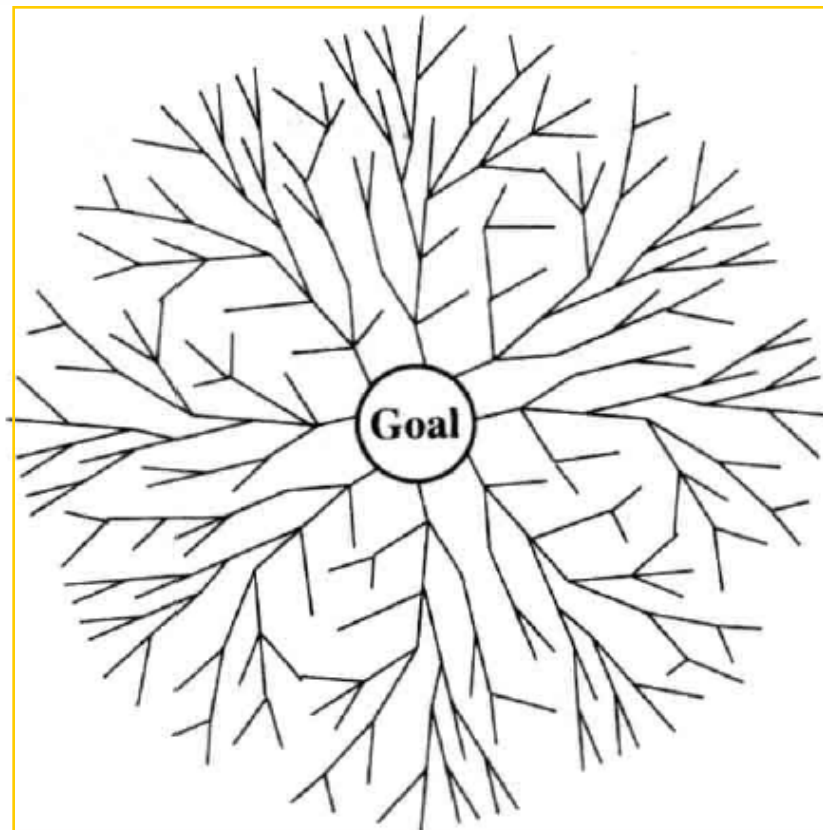
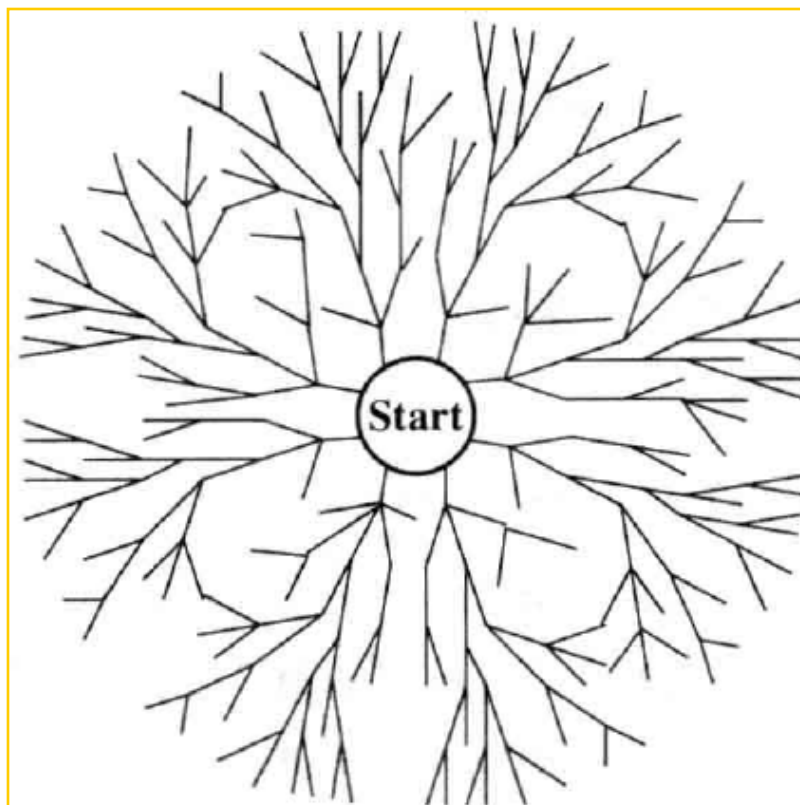


图 3.20 即将成功的双向搜索的示意图，从起始结点发出的一个分支与从目标结点发出的一个分支即将相遇

双向最佳

优先搜索

function BiBF-SEARCH($problem_F, f_F, problem_B, f_B$) **returns** 一个解节点或 *failure*

$node_F \leftarrow \text{NODE}(problem_F.INITIAL)$ // 初始状态节点

$node_B \leftarrow \text{NODE}(problem_B.INITIAL)$ // 目标状态节点

$frontier_F \leftarrow$ 按 f_F 排序的优先队列, 其中一个元素为 $node_F$

$frontier_B \leftarrow$ 按 f_B 排序的优先队列, 其中一个元素为 $node_B$

$reached_F \leftarrow$ 一个查找表, 其中一个条目的键是 $node_F.STATE$ 且值是 $node_F$

$reached_B \leftarrow$ 一个查找表, 其中一个条目的键是 $node_B.STATE$ 且值是 $node_B$

$solution \leftarrow failure$

while not TERMINATED($solution, frontier_F, frontier_B$) **do**

if $f_F(\text{TOP}(frontier_F)) < f_B(\text{TOP}(frontier_B))$ **then**

$solution \leftarrow \text{PROCEED}(F, problem_F, frontier_F, reached_F, reached_B, solution)$

else $solution \leftarrow \text{PROCEED}(B, problem_B, frontier_B, reached_B, reached_F, solution)$

return $solution$

function PROCEED($dir, problem, frontier, reached, reached_2, solution$) **returns** 一个解

// 在 $frontier$ 上扩展节点, 对照 $reached_2$ 中的另一个边界

// 变量 dir 是方向: 要么是 F (代表正向), 要么是 B (代表反向)

$node \leftarrow \text{POP}(frontier)$

for each $child$ **in** EXPAND($problem, node$) **do**

$s \leftarrow child.STATE$

if s 不在 $reached$ 中 **or** $\text{PATH-COST}(child) < \text{PATH-COST}(reached[s])$ **then**

$reached[s] \leftarrow child$

将 $child$ 添加到 $frontier$

if s 不在 $reached_2$ 中 **then**

$solution_2 \leftarrow \text{JOIN-NODES}(dir, child, reached_2[s])$

if $\text{PATH-COST}(solution_2) < \text{PATH-COST}(solution)$ **then**

$solution \leftarrow solution_2$

return $solution$

无信息搜索策略对比

指标	广度优先	一致代价	深度优先	深度受限	迭代加深	双向 (如适用)
完备性	是 ¹	是 ^{1,2}	否	否	是 ¹	是 ^{1,4}
代价最优	是 ³	是	否	否	是 ³	是 ^{3,4}
时间复杂性	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
空间复杂性	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$

注: ¹ 如果 b 是有限的且状态空间要么有解要么有限, 则算法是完备的。 ² 如果所有动作的代价都 $\geq \epsilon > 0$, 算法是完备的。 ³ 如果动作代价都相同, 算法是代价最优的。 ⁴ 如果两个方向均使用广度优先搜索或一致代价搜索